

Manual de arquitectura – Mercy Food

Versión: 1.0

Fecha: 30/oct/2025

1. Introducción

Este manual de arquitectura proporciona una descripción detallada de la estructura técnica, las decisiones de diseño y las tecnologías utilizadas en el desarrollo de la plataforma web **Mercy Food**. El objetivo es servir como una guía central para el equipo de desarrollo, asegurando la coherencia y calidad del sistema a lo largo de su ciclo de vida.

2. Atributos no funcionales y de calidad

Los siguientes atributos de calidad fueron los pilares para las decisiones de arquitectura del sistema:

- **Rendimiento:** La aplicación debe ser rápida y responsiva. Las consultas a la base de datos están optimizadas para que las cargas de menús y restaurantes sean inferiores a 3 segundos.
- **Usabilidad:** La interfaz debe ser intuitiva y fácil de usar para los tres perfiles de usuario, minimizando la curva de aprendizaje.
- **Seguridad:** Se prioriza la protección de los datos del usuario. Las contraseñas se almacenan cifradas mediante hashing y se utiliza protección CSRF en todos los formularios para prevenir ataques.
- **Escalabilidad:** Aunque la versión inicial utiliza SQLite para simplificar el desarrollo, la arquitectura basada en Laravel permite una transición sencilla a sistemas de bases de datos más robustos como MySQL o PostgreSQL para soportar un futuro crecimiento de usuarios y transacciones.
- **Mantenibilidad:** El código está organizado siguiendo estrictamente el patrón MVC, lo que facilita la localización de errores, la adición de nuevas funcionalidades y la actualización de componentes de forma aislada.

3. Modelo arquitectónico

3.1. Patrón Modelo-Vista-Controlador (MVC)

Mercy Food está construido sobre el patrón de diseño **Modelo-Vista-Controlador**, que es el núcleo del framework Laravel. Esta elección garantiza una separación clara de responsabilidades:

- **Modelo (M):** Representa la capa de datos y la lógica de negocio. En nuestro proyecto, los modelos como User.php, Pedido.php y Platillo.php interactúan directamente con la base de datos a través del ORM Eloquent. Son responsables de definir las relaciones, obtener y guardar la información.
- **Vista (V):** Es la capa de presentación, responsable de mostrar la interfaz de usuario. En Mercy Food, las vistas son archivos *.blade.php que renderizan el HTML que el usuario ve en su navegador. No contienen lógica de negocio compleja, solo presentan los datos que les pasa el controlador.
- **Controlador (C):** Actúa como el intermediario entre el Modelo y la Vista. Recibe las peticiones del usuario a través de las rutas, solicita los datos necesarios al Modelo, procesa esa información y finalmente la envía a la Vista adecuada para ser mostrada. Ejemplos clave son PedidoController.php y PlatilloController.php.

Este patrón es fundamental para la mantenibilidad y organización del proyecto.

4. Tecnología y herramientas

El ecosistema tecnológico de Mercy Food se seleccionó para maximizar la productividad y la robustez.

Componente	Tecnología / Herramienta	Versión	Justificación
Backend Framework	Laravel	11.x	Framework de PHP robusto, seguro y con un ecosistema muy completo (ORM Eloquent, motor de plantillas Blade, sistema de rutas) que acelera el desarrollo.
Lenguaje principal	PHP	8.2+	Lenguaje maduro, ampliamente soportado y optimizado para el desarrollo web.
Base de Datos	SQLite	3.x	Base de datos basada en archivos, ideal para el desarrollo y prototipado por su simplicidad (no requiere un servidor de BD) y fácil configuración.
Frontend	Blade & CSS & JS	N/A	Se utilizan las plantillas Blade de Laravel para el renderizado del HTML, junto con archivos CSS y JavaScript planos para el estilo y la interactividad, evitando la complejidad de un framework de frontend.
Gestor de Dependencias	Composer	2.x	Herramienta estándar para la gestión de paquetes y librerías en el ecosistema PHP.
Servidor de Desarrollo	Artisan Serve	N/A	Servidor web ligero integrado en Laravel, perfecto para el entorno de desarrollo local.

5. Justificación de la arquitectura

La elección de **Laravel** como framework principal fue una decisión estratégica. Su arquitectura **MVC** no solo impone una estructura de código limpia y organizada, sino que también ofrece un conjunto de herramientas integradas que resuelven problemas comunes del desarrollo web, como la autenticación, el enrutamiento y la interacción con la base de datos.

El uso de **SQLite** en la fase de desarrollo permitió agilizar la configuración inicial del proyecto, eliminando la necesidad de instalar y gestionar un servidor de base de datos como MySQL. Esto hace que el proyecto sea más portable y fácil de clonar y ejecutar para nuevos desarrolladores. La arquitectura está preparada para cambiar a una base de datos más potente con solo modificar unas pocas líneas en el archivo de configuración .env, demostrando la flexibilidad del diseño.

6. Control de versiones

Todo el código fuente y la documentación del proyecto se gestionan utilizando **Git**. El repositorio central está alojado en **GitHub**, lo que facilita la colaboración, el seguimiento de cambios y la integración continua. Se sigue un flujo de trabajo estándar basado en commits descriptivos para mantener un historial claro y comprensible del desarrollo del proyecto.

7. Diseño de la base de datos

El diseño de la base de datos se refleja en el **Diagrama Entidad-Relación (E-R)**. Este diagrama visualiza las tablas principales del sistema y las relaciones que existen entre ellas, como la relación uno a muchos entre un usuario y sus pedidos.

